

XI. Advanced Topics

Sum of Integers 1 through N

What is $1 + 2 + \dots + n$? Let's figure it out by pairing up low values with high values.

If n is even, we pair 1 with n , 2 with $n - 1$, and so on. We will have $\frac{n}{2}$ pairs each with sum $n + 1$.

If n is odd, we pair 0 with n , 1 with $n - 1$, and so on. We will have $\frac{n+1}{2}$ pairs with sum n .

n is even			
pair #	a	b	a + b
1	1	n	n + 1
2	2	n - 1	n + 1
3	3	n - 2	n + 1
4	4	n - 3	n + 1
...	...	...	...
$\frac{n}{2}$	$\frac{n}{2}$	$\frac{n}{2} + 1$	n + 1
total:	$\frac{n}{2} * (n + 1)$		

n is odd			
pair #	a	b	a + b
1	0	n	n
2	1	n - 1	n
3	2	n - 2	n
4	3	n - 3	n
...	...	...	...
$\frac{n+1}{2}$	$\frac{n-1}{2}$	$\frac{n+1}{2}$	n
total:	$\frac{n+1}{2} * n$		

In either case, the sum is $\frac{n(n+1)}{2}$.

This reasoning comes up a lot in nested loops. For example, consider the following code:

```
1 for (int i = 0; i < n; i++) {
2     for (int j = i + 1; j < n; j++) {
3         System.out.println(i + j);
4     }
5 }
```

On the first iteration of the outer for loop, the inner for loop iterates $n - 1$ times. On the second iteration of the outer for loop, the inner for loop iterates $n - 2$ times. Next, $n - 3$, then $n - 4$, and so on. There are $\frac{n(n-1)}{2}$ total iterations of the inner for loop. Therefore, this code takes $O(n^2)$ time.

Sum of Powers of 2

Consider this sequence: $2^0 + 2^1 + 2^2 + \dots + 2^n$. What is its result?

A nice way to see this is by looking at these values in binary.

	Power	Binary	Decimal
	2^0	00001	1
	2^1	00010	2
	2^2	00100	4
	2^3	01000	8
	2^4	10000	16
sum:	$2^5 - 1$	11111	$32 - 1 = 31$

Therefore, the sum of $2^0 + 2^1 + 2^2 + \dots + 2^n$ would, in base 2, be a sequence of $(n + 1)$ 1s. This is $2^{n+1} - 1$.

Takeaway: The sum of a sequence of powers of two is roughly equal to the next value in the sequence.

Bases of Logs

Suppose we have something in $\log_2$ (log base 2). How do we convert that to $\log_{10}$? That is, what's the relationship between $\log_b k$ and $\log_x k$?

Let's do some math. Assume $c = \log_b k$ and $y = \log_x k$.

```
log_b k = c --> b^c = k           // This is the definition of log.
log_x(b^c) = log_x k             // Take log of both sides of b^c = k.
c log_x b = log_x k              // Rules of logs. You can move out the exponents.
c = log_b k = log_x k / log_x b  // Dividing above expression and substituting c.
```

Therefore, if we want to convert $\log_2 p$ to $\log_{10} p$, we just do this:

$$\log_{10} p = \frac{\log_2 p}{\log_2 10}$$

Takeaway: Logs of different bases are only off by a constant factor. For this reason, we largely ignore what the base of a log within a big O expression. It doesn't matter since we drop constants anyway.

Permutations

How many ways are there of rearranging a string of n unique characters? Well, you have n options for what to put in the first characters, then $n - 1$ options for what to put in the second slot (one option is taken), then $n - 2$ options for what to put in the third slot, and so on. Therefore, the total number of strings is $n!$.

$$n! = n * (n - 1) * (n - 2) * (n - 3) * \dots * 1$$

What if you were forming a k -length string (with all unique characters) from n total unique characters? You can follow similar logic, but you'd just stop your selection/multiplication earlier.

$$\frac{n!}{(n-k)!} = n * (n - 1) * (n - 2) * (n - 3) * \dots * (n - k + 1)$$

Combinations

Suppose you have a set of n distinct characters. How many ways are there of selecting k characters into a new set (where order doesn't matter)? That is, how many k -sized subsets are there out of n distinct elements? This is what the expression n -choose- k means, which is often written $\binom{n}{k}$.

Imagine we made a list of all the sets by first writing all k -length substrings and then taking out the duplicates.

From the above *Permutations* section, we'd have $\frac{n!}{(n-k)!}$ k -length substrings.

Since each k -sized subset can be rearranged $k!$ unique ways into a string, each subset will be duplicated $k!$ times in this list of substrings. Therefore, we need to divide by $k!$ to take out these duplicates.

$$\binom{n}{k} = \frac{1}{k!} * \frac{n!}{(n-k)!} = \frac{n!}{k!(n-k)!}$$

Proof by Induction

Induction is a way of proving something to be true. It is closely related to recursion. It takes the following form.

Task: Prove statement $P(k)$ is true for all $k \geq b$.

- Base Case: Prove the statement is true for $P(b)$. This is usually just a matter of plugging in numbers.
- Assumption: Assume the statement is true for $P(n)$.
- Inductive Step: Prove that if the statement is true for $P(n)$, then it's true for $P(n+1)$.

This is like dominoes. If the first domino falls, and one domino always knocks over the next one, then all the dominoes must fall.

Let's use this to prove that there are 2^n subsets of an n -element set.

- Definitions: let $S = \{a_1, a_2, a_3, \dots, a_n\}$ be the n -element set.

XI. Advanced Topics

- Base case: Prove there are 2^0 subsets of $\{\}$. This is true, since the only subset of $\{\}$ is $\{\}$.
- Assume that there are 2^n subsets of $\{a_1, a_2, a_3, \dots, a_n\}$.
- Prove that there are 2^{n+1} subsets of $\{a_1, a_2, a_3, \dots, a_{n+1}\}$.

Consider the subsets of $\{a_1, a_2, a_3, \dots, a_{n+1}\}$. Exactly half will contain a_{n+1} and half will not.

The subsets that do not contain a_{n+1} are just the subsets of $\{a_1, a_2, a_3, \dots, a_n\}$. We assumed there are 2^n of those.

Since we have the same number of subsets with x as without x , there are 2^n subsets with a_{n+1} .

Therefore, we have $2^n + 2^n$ subsets, which is 2^{n+1} .

Many recursive algorithms can be proved valid with induction.

► Topological Sort

A topological sort of a directed graph is a way of ordering the list of nodes such that if (a, b) is an edge in the graph then a will appear before b in the list. If a graph has cycles or is not directed, then there is no topological sort.

There are a number of applications for this. For example, suppose the graph represents parts on an assembly line. The edge (Handle, Door) indicates that you need to assemble the handle before the door. The topological sort would offer a valid ordering for the assembly line.

We can construct a topological sort with the following approach.

1. Identify all nodes with no incoming edges and add those nodes to our topological sort.
 - » We know those nodes are safe to add first since they have nothing that needs to come before them. Might as well get them over with!
 - » We know that such a node must exist if there's no cycle. After all, if we picked an arbitrary node we could just walk edges backwards arbitrarily. We'll either stop at some point (in which case we've found a node with no incoming edges) or we'll return to a prior node (in which case there is a cycle).
2. When we do the above, remove each node's outbound edges from the graph.
 - » Those nodes have already been added to the topological sort, so they're basically irrelevant. We can't violate those edges anymore.
3. Repeat the above, adding nodes with no incoming edges and removing their outbound edges. When all the nodes have been added to the topological sort, then we are done.

More formally, the algorithm is this:

1. Create a queue `order`, which will eventually store the valid topological sort. It is currently empty.
2. Create a queue `processNext`. This queue will store the next nodes to process.
3. Count the number of incoming edges of each node and set a class variable `node.inbound`. Nodes typically only store their outgoing edges. However, you can count the inbound edges by walking through each node n and, for each of its outgoing edges (n, x) , incrementing $x.inbound$.
4. Walk through the nodes again and add to `processNext` any node where $x.inbound == 0$.
5. While `processNext` is not empty, do the following:
 - » Remove first node n from `processNext`.

- » Foreach edge (n, x) , decrement $x.inbound$. If $x.inbound == 0$, append x to `processNext`.
- » Append n to `order`.

6. If `order` contains all the nodes, then it has succeeded. Otherwise, the topological sort has failed due to a cycle.

This algorithm does sometimes come up in interview questions. Your interviewer probably wouldn't expect you to know it offhand. However, it would be reasonable to have you derive it even if you've never seen it before.

► Dijkstra's Algorithm

In some graphs, we might want to have edges with weights. If the graph represented cities, each edge might represent a road and its weight might represent the travel time. In this case, we might want to ask, just as your GPS mapping system does, what's the shortest path from your current location to another point p ? This is where Dijkstra's algorithm comes in.

Dijkstra's algorithm is a way to find the shortest path between two points in a weighted directed graph (which might have cycles). All edges must have positive values.

Rather than just stating what Dijkstra's algorithm is, let's try to derive it. Consider the earlier described graph. We could find the shortest path from s to t by literally taking all possible routes using actual time. (Oh, and we'll need a machine to clone ourselves.)

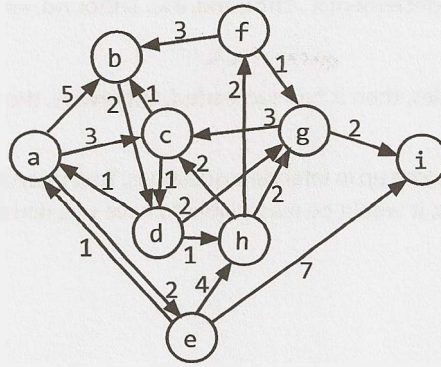
1. Start off at s .
2. For each of s 's outbound edges, clone ourselves and start walking. If the edge (s, x) has weight 5, we should actually take 5 minutes to get there.
3. Each time we get to a node, check if anyone's been there before. If so, then just stop. We're automatically not as fast as another path since someone beat us here from s . If no one has been here before, then clone ourselves and head out in all possible directions.
4. The first one to get to t wins.

This works just fine. But, of course, in the real algorithm we don't want to literally use a timer to find the shortest path.

Imagine that each clone could jump immediately from one node to its adjacent nodes (regardless of the edge weight), but it kept a `time_so_far` log of how long its path would have taken if it did walk at the "true" speed. Additionally, only one person moves at a time, and it's always the one with the lowest `time_so_far`. This is sort of how Dijkstra's algorithm works.

Dijkstra's algorithm finds the minimum weight path from a start node s to every node on the graph.

Consider the following graph.



Assume we are trying to find the shortest path from a to i. We'll use Dijkstra's algorithm to find the shortest path from a to all other nodes, from which we will clearly have the shortest path from a to i.

We first initialize several variables:

- `path_weight[node]`: maps from each node to the total weight of the shortest path. All values are initialized to infinity, except for `path_weight[a]` which is initialized to 0.
- `previous[node]`: maps from each node to the previous node in the (current) shortest path.
- `remaining`: a priority queue of all nodes in the graph, where each node's priority is defined by its `path_weight`.

Once we've initialized these values, we can start adjusting the values of `path_weight`.

A (min) **priority queue** is an abstract data type that—at least in this case—supports insertion of an object and key, removing the object with the minimum key, and decreasing a key. (Think of it like a typical queue, except that, instead of removing the oldest item, it removes the item with the lowest or highest priority.) It is an abstract data type because it is defined by its behavior (its operations). Its underlying implementation can vary. You could implement a priority queue with an array or a min (or max) heap (or many other data structures).

We iterate through the nodes in `remaining` (until `remaining` is empty), doing the following:

1. Select the node in `remaining` with the lowest value in `path_weight`. Call this node `n`.
2. For each adjacent node, compare `path_weight[x]` (which is the weight of the current shortest path from a to `x`) to `path_weight[n] + edge_weight(n, x)`. That is, could we get a path from a to `x` with lower weight by going through `n` instead of our current path? If so, update `path_weight` and `previous`.
3. Remove `n` from `remaining`.

When `remaining` is empty, then `path_weight` stores the weight of the current shortest path from a to each node. We can reconstruct this path by tracing through `previous`.

Let's walk through this on the above graph.

1. The first value of `n` is a. We look at its adjacent nodes (b, c, and e), update the values of `path_weight` (to 5, 3, and 2) and `previous` (to a) and then remove a from `remaining`.
2. Then, we go to the next smallest node, which is e. We previously updated `path_weight[e]` to be 2. Its adjacent nodes are h and i, so we update `path_weight` (to 6 and 9) and `previous` for both of those.

Observe that 6 is `path_weight[e]` (which is 2) + the weight of the edge (e, h) (which is 4).

- The next smallest node is c, which has `path_weight` 3. Its adjacent nodes are b and d. The value of `path_weight[d]` is infinity, so we update it to 4 (which is `path_weight[c] + weight(edge c, d)`). The value of `path_weight[b]` has been previously set to 5. However, since `path_weight[c] + weight(edge c, b)` (which is $3 + 1 = 4$) is less than 5, we update `path_weight[b]` to 4 and previous to c. This indicates that we would improve the path from a to b by going through c.

We continued doing this until `remaining` is empty. The following diagram shows the changes to the `path_weight` (left) and `previous` (right) at each step. The topmost row shows the current value for n (the node we are removing from `remaining`). We black out a row after it has been removed from `remaining`.

	INITIAL		n = a		n = e		n = c		n = b		n = d		n = h		n = g		n = f		FINAL	
	wt	pr	wt	pr	wt	pr	wt	pr	wt	pr	wt	pr	wt	pr	wt	pr	wt	pr	wt	pr
a	0	-																	0	-
b	∞	-	5	a			4	c											4	c
c	∞	-	3	a															3	a
d	∞	-					4	c											4	c
e	∞	-	2	a															2	a
f	∞	-											7	h					7	h
g	∞	-									6	d							6	d
h	∞	-			6	e					5	d							5	d
i	∞	-	∞	-	9	e									8	g			8	g

Once we're done, we can follow this chart backwards, starting at i to find the actual path. In this case, the smallest weight path has weight 8 and is a -> c -> d -> g -> i.

Priority Queue and Runtime

As mentioned earlier, our algorithm used a priority queue, but this data structure can be implemented in different ways.

The runtime of this algorithm depends heavily on the implementation of the priority queue. Assume you have v vertices and e nodes.

- If you implemented the priority queue with an array, then you would call `remove_min` up to v times. Each operation would take $O(v)$ time, so you'd spend $O(v^2)$ time in the `remove_min` calls. Additionally, you would update the values of `path_weight` and `previous` at most once per edge, so that's $O(e)$ time doing those updates. Observe that e must be less than or equal to v^2 since you can't have more edges than there are pairs of vertices. Therefore, the total runtime is $O(v^2)$.
- If you implemented the priority queue with a min heap, then the `remove_min` calls will each take $O(\log v)$ time (as will inserting and updating a key). We will do one `remove_min` call for each vertex, so that's $O(v \log v)$ (v vertices at $O(\log v)$ time each). Additionally, on each edge, we might call one update key or insert operation, so that's $O(e \log v)$. The total runtime is $O((v + e) \log v)$.

Which one is better? Well, that depends. If the graph has a lot of edges, then v^2 will be close to e . In this case, you might be better off with the array implementation, as $O(v^2)$ is better than $O((v + v^2) \log v)$. However, if the graph is sparse, then e is much less than v^2 . In this case, the min heap implementation may be better.

► Hash Table Collision Resolution

Essentially any hash table can have collisions. There are a number of ways of handling this.

Chaining with Linked Lists

With this approach (which is the most common), the hash table's array maps to a linked list of items. We just add items to this linked list. As long as the number of collisions is fairly small, this will be quite efficient.

In the worst case, lookup is $O(n)$, where n is the number of elements in the hash table. This would only happen with either some very strange data or a very poor hash function (or both).

Chaining with Binary Search Trees

Rather than storing collisions in a linked list, we could store collisions in a binary search tree. This will bring the worst-case runtime to $O(\log n)$.

In practice, we would rarely take this approach unless we expected an extremely nonuniform distribution.

Open Addressing with Linear Probing

In this approach, when a collision occurs (there is already an item stored at the designated index), we just move on to the next index in the array until we find an open spot. (Or, sometimes, some other fixed distance, like the `index + 5`.)

If the number of collisions is low, this is a very fast and space-efficient solution.

One obvious drawback of this is that the total number of entries in the hash table is limited by the size of the array. This is not the case with chaining.

There's another issue here. Consider a hash table with an underlying array of size 100 where indexes 20 through 29 are filled (and nothing else). What are the odds of the next insertion going to index 30? The odds are 10% because an item mapped to any index between 20 and 29 will wind up at index 30. This causes an issue called *clustering*.

Quadratic Probing and Double Hashing

The distance between probes does not need to be linear. You could, for example, increase the probe distance quadratically. Or, you could use a second hash function to determine the probe distance.

► Rabin-Karp Substring Search

The brute force way to search for a substring S in a larger string B takes $O(s(b-s))$ time, where s is the length of S and b is the length of B . We do this by searching through the first $b - s + 1$ characters in B and, for each, checking if the next s characters match S .

The Rabin-Karp algorithm optimizes this with a little trick: if two strings are the same, they must have the same hash value. (The converse, however, is not true. Two different strings can have the same hash value.)

Therefore, if we efficiently precompute a hash value for each sequence of s characters within B , we can find the locations of S in $O(b)$ time. We then just need to validate that those locations really do match S .

For example, imagine our hash function was simply the sum of each character (where space = 0, a = 1, b = 2, and so on). If S is `ear` and B = `doe are hearing me`, we'd then just be looking for sequences where the sum is 24 ($e + a + r$). This happens three times. For each of those locations, we'd check if the string really is `ear`.

char:	d	o	e		a	r	e		h	e	a	r	i	n	g		m	e
code:	4	15	5	0	1	18	5	0	8	5	1	18	9	14	7	0	13	5
sum of next 3:	24	20	6	19	24	23	13	13	14	24	28	41	30	21	20	18		

If we computed these sums by doing $\text{hash}(\text{'doe'})$, then $\text{hash}(\text{'oe'})$, then $\text{hash}(\text{'e a'})$, and so on, we would still be at $O(s(b-s))$ time.

Instead, we compute the hash values by recognizing that $\text{hash}(\text{'oe'}) = \text{hash}(\text{'doe'}) - \text{code}(\text{'d'}) + \text{code}(\text{' '})$. This takes $O(b)$ time to compute all the hashes.

You might argue that, still, in the worst case this will take $O(s(b-s))$ time since many of the hash values could match. That's absolutely true—for this hash function.

In practice, we would use a better *rolling hash function*, such as the Rabin fingerprint. This essentially treats a string like `doe` as a base 128 (or however many characters are in our alphabet) number.

$$\text{hash}(\text{'doe'}) = \text{code}(\text{'d'}) * 128^2 + \text{code}(\text{'o'}) * 128^1 + \text{code}(\text{'e'}) * 128^0$$

This hash function will allow us to remove the `d`, shift the `o` and `e`, and then add in the space.

$$\text{hash}(\text{'oe '}) = (\text{hash}(\text{'doe'}) - \text{code}(\text{'d'}) * 128^2) * 128 + \text{code}(\text{' '})$$

This will considerably cut down on the number of false matches. Using a good hash function like this will give us expected time complexity of $O(s + b)$, although the worst case is $O(sb)$.

Usage of this algorithm comes up fairly frequently in interviews, so it's useful to know that you can identify substrings in linear time.

► AVL Trees

An AVL tree is one of two common ways to implement tree balancing. We will only discuss insertions here, but you can look up deletions separately if you're interested.

Properties

An AVL tree stores in each node the height of the subtrees rooted at this node. Then, for any node, we can check if it is height balanced: that the height of the left subtree and the height of the right subtree differ by no more than one. This prevents situations where the tree gets too lopsided.

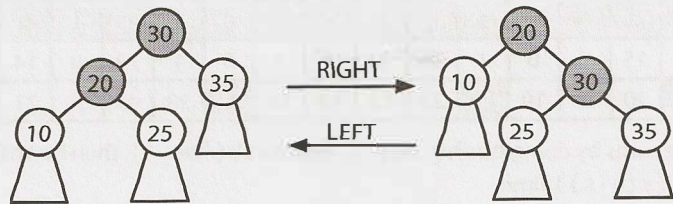
$$\text{balance}(n) = n.\text{left}.\text{height} - n.\text{right}.\text{height}$$

$$-1 \leq \text{balance}(n) \leq 1$$

Inserts

When you insert a node, the balance of some nodes might change to -2 or 2. Therefore, when we "unwind" the recursive stack, we check and fix the balance at each node. We do this through a series of rotations.

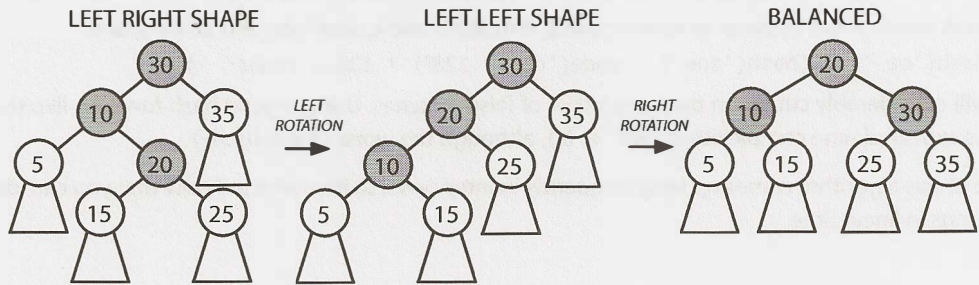
Rotations can be either left or right rotations. The right rotation is an inverse of the left rotation.



Depending on the balance and where the imbalance occurs, we fix it in a different way.

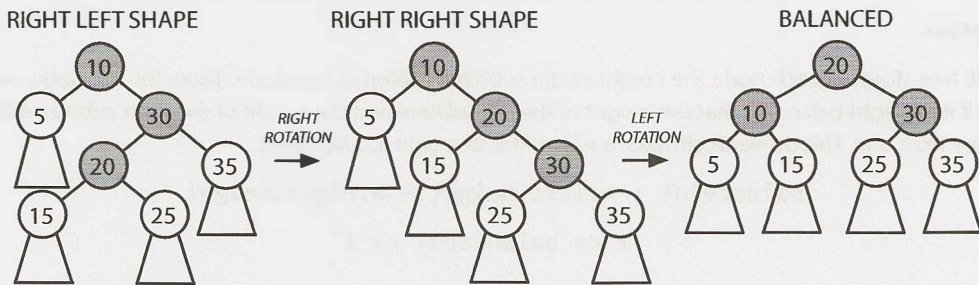
• Case 1: Balance is 2.

In this case, the left's height is two bigger than the right's height. If the left side is larger, the left subtree's extra nodes must be hanging to the left (as in LEFT LEFT SHAPE) or hanging to the right (as in LEFT RIGHT SHAPE). If it looks like the LEFT RIGHT SHAPE, transform it with the rotations below into the LEFT LEFT SHAPE then into BALANCED. If it looks like the LEFT LEFT SHAPE already, just transform it into BALANCED.



• Case 2: Balance is -2.

This case is the mirror image of the prior case. The tree will look like either the RIGHT LEFT SHAPE or the RIGHT RIGHT SHAPE. Perform the rotations below to transform it into BALANCED.



In both cases, "balanced" just means that the balance of the tree is between -1 and 1. It does not mean that the balance is 0.

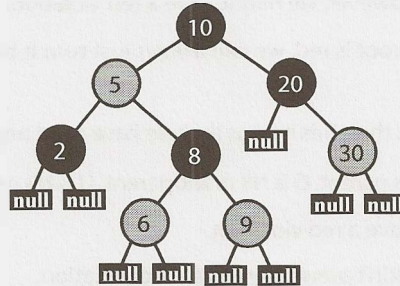
We recurse up the tree, fixing any imbalances. If we ever achieve a balance of 0 on a subtree, then we know that we have completed all the balances. This portion of the tree will not cause another, higher subtree to have a balance of -2 or 2. If we were doing this non-recursively, then we could break from the loop.

► Red-Black Trees

Red-black trees (a type of self-balancing binary search tree) do not ensure quite as strict balancing, but the balancing is still good enough to ensure $O(\log N)$ insertions, deletions, and retrievals. They require a bit less memory and can rebalance faster (which means faster insertions and removals), so they are often used in situations where the tree will be modified frequently.

Red-black trees operate by enforcing a quasi-alternating red and black coloring (under certain rules, described below) and then requiring every path from a node to its leaves to have the same number of black nodes. Doing so leads to a reasonably balanced tree.

The tree below is a red-black tree (where the red nodes are indicated with gray):



Properties

1. Every node is either red or black.
2. The root is black.
3. The leaves, which are NULL nodes, are considered black.
4. Every red node must have two black children. That is, a red node cannot have red children (although a black node can have black children).
5. Every path from a node to its leaves must have the same number of black children.

Why It Balances

Property #4 means that two red nodes cannot be adjacent in a path (e.g., parent and child). Therefore, no more than half the nodes in a path can be red.

Consider two paths from a node (say, the root) to its leaves. The paths must have the same number of black nodes (property #5), so let's assume that their red node counts are as different as possible: one path contains the minimum number of red nodes and the other one contains the maximum number.

- Path 1 (Min Red): The minimum number of red nodes is zero. Therefore, path 1 has b nodes total.
- Path 2 (Max Red): The maximum number of red nodes is b , since red nodes must have black children and there are b black nodes. Therefore, path 2 has $2b$ nodes total.

Therefore, even in the most extreme case, the lengths of paths cannot differ by more than a factor of two. That's good enough to ensure an $O(\log N)$ find and insert runtime.

If we can maintain these properties, we'll have a (sufficiently) balanced tree—good enough to ensure $O(\log N)$ insert and find, anyway. The question then is how to maintain these properties efficiently. We'll only discuss insertion here, but you can look up deletion on your own.

Insertion

Inserting a new node into a red-black tree starts off with a typical binary search tree insertion.

- New nodes are inserted at a leaf, which means that they replace a black node.
- New nodes are always colored red and are given two black leaf (NULL) nodes.

Once we've done that, we fix any resulting red-black property violations. We have two possible violations:

- Red violations: A red node has a red child (or the root is red).
- Black violations: One path has more blacks than another path.

The node inserted is red. We didn't change the number of black nodes on any path to a leaf, so we know that we won't have a black violation. However, we might have a red violation.

In the special case that where the root is red, we can always just turn it black to satisfy property 2, without violating the other constraints.

Otherwise, if there's a red violation, then this means that we have a red node under another red node. Oops!

Let's call N the current node. P is N's parent. G is N's grandparent. U is N's uncle and P's sibling. We know that:

- N is red and P is red, since we have a red violation.
- G is definitely black, since we didn't *previously* have a red violation.

The unknown parts are:

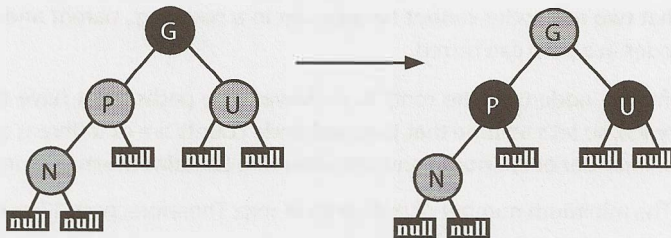
- U could be either red or black.
- U could be either a left or right child.
- N could be either a left or right child.

By simple combinatorics, that's eight cases to consider. Fortunately some of these cases will be equivalent.

• **Case 1: U is red.**

It doesn't matter whether U is a left or right child, nor whether P is a left or right child. We can merge four of our eight cases into one.

If U is red, we can just toggle the colors of P, U, and G. Flip G from black to red. Flip P and U from red to black. We haven't changed the number of black nodes in any path.



However, by making G red, we might have created a red violation with G's parent. If so, we recursively apply the full logic to handle a red violation, where this G becomes the new N.

Note that in the general recursive case, N, P, and U may also have subtrees in place of each black NULL (the leaves shown). In Case 1, these subtrees stay attached to the same parents, as the tree structure remains unchanged.

• **Case 2: U is black.**

We'll need to consider the configurations (left vs. right child) of N and U. In each case, our goal is to fix up the red violation (red on top of red) without:

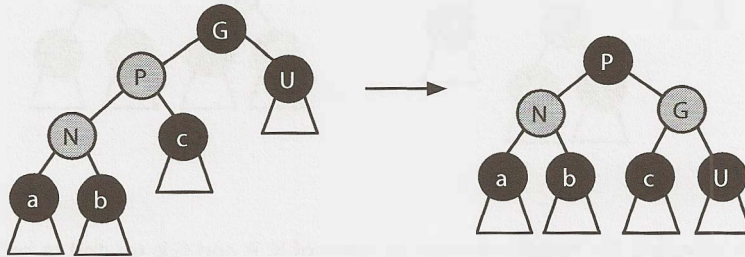
- » Messing up the ordering of the binary search tree.
- » *Introducing a black violation (more black nodes on one path than another).*

If we can do this, we're good. In each of the cases below, the red violation is fixed with rotations that maintain the node ordering.

Further, the below rotations maintain the exact number of black nodes in each path through the affected portion of the tree that were in place beforehand. The children of the rotating section are either NULL leaves or subtrees that remain internally unchanged.

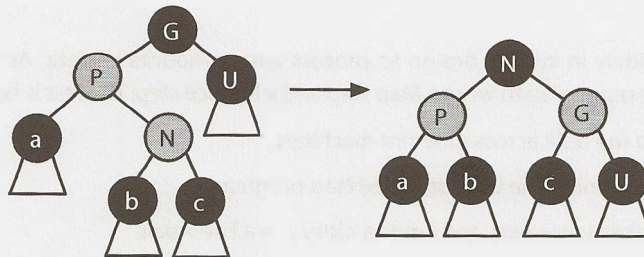
Case A: N and P are both left children.

We resolve the red violation with the rotation of N, P, and G and the associated recoloring shown below. If you picture the in-order traversal, you can see the rotation maintains the node ordering ($a \leq N \leq b \leq P \leq c \leq G \leq U$). The tree maintains the same, equal number of black nodes in the path down to each subtree a, b, c, and U (which may all be NULL).



Case B: P is a left child, and N is a right child.

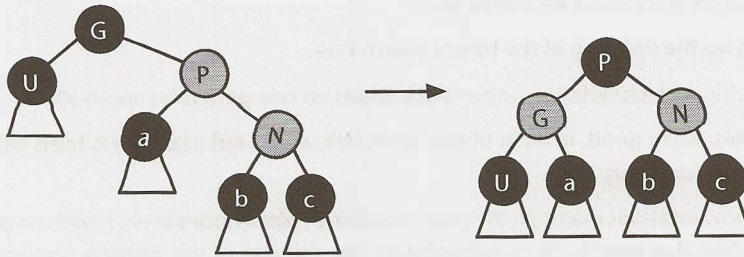
The rotations in Case B resolve the red violation and maintain the in-order property: $a \leq P \leq b \leq N \leq c \leq G \leq U$. Again, the count of the black nodes remains constant in each path down to the leaves (or subtrees).



XI. Advanced Topics

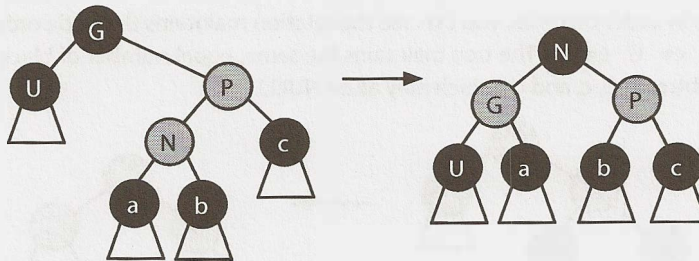
Case C: N and P are both right children.

This is a mirror image of case A.



Case D: N is a left child, and P is a right child.

This is a mirror image of case B.



In each of Case 2's subcases, the middle element by value of N, P, and G is rotated to become the root of what was G's subtree, and that element and G swap colors.

That said, do not try to just memorize these cases. Rather, study why they work. How does each one ensure no red violations, no black violations, and no violations of the binary search tree property?

► MapReduce

MapReduce is used widely in system design to process large amounts of data. As its name suggests, a MapReduce program requires you to write a Map step and a Reduce step. The rest is handled by the system.

1. The system splits up the data across different machines.
2. Each machine starts running the user-provided Map program.
3. The Map program takes some data and emits a `<key, value>` pair.
4. The system-provided `Shuffle` process reorganizes the data so that all `<key, value>` pairs associated with a given key go to the same machine, to be processed by `Reduce`.
5. The user-provided `Reduce` program takes a key and a set of associated values and “reduces” them in some way, emitting a new key and value. The results of this might be fed back into the `Reduce` program for more reducing.

The typical example of using MapReduce—basically the “Hello World” of MapReduce—is counting the frequency of words within a set of documents.

Of course, you could write this as a single function that reads in all the data, counts the number of times each word appears via a hash table, and then outputs the result.

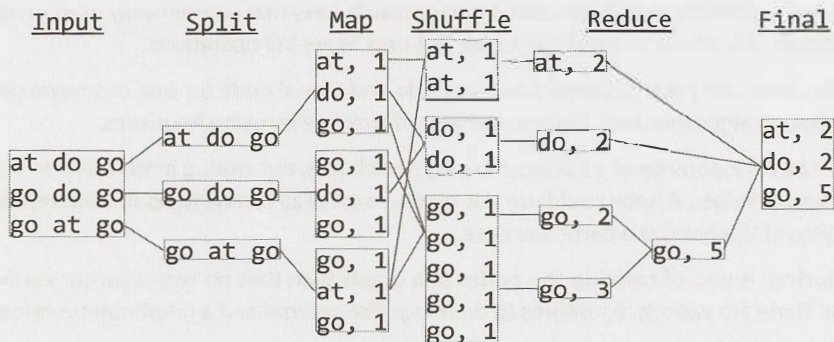
MapReduce allows you to process the document in parallel. The Map function reads in a document and emits just each individual word and the count (which is always 1). The Reduce function reads in keys (words) and associated values (counts). It emits the sum of the counts. This sum could possibly wind up as input for another call to Reduce on the same key (as shown in the diagram).

```

1 void map(String name, String document):
2     for each word w in document:
3         emit(w, 1)
4
5 void reduce(String word, Iterator partialCounts):
6     int sum = 0
7     for each count in partialCounts:
8         sum += count
9     emit(word, sum)

```

The diagram below shows how this might work on this example.



Here's another example: You have a list of data in the form {City, Temperature, Date}. Calculate the average temperature in each city every year. For example {(2012, Philadelphia, 58.2), (2011, Philadelphia, 56.6), (2012, Seattle, 45.1)}.

- **Map:** The Map step outputs a key value pair where the key is City_Year and the value is (Temperature, 1). The '1' reflects that this is the average temperature out of one data point. This will be important for the Reduce step.
- **Reduce:** The Reduce step will be given a list of temperatures that correspond with a particular city and year. It must use these to compute the average temperature for this input. You cannot simply add up the temperatures and divide by the number of values.

To see this, imagine we have five data points for a particular city and year: 25, 100, 75, 85, 50. The Reduce step might only get some of this data at once. If you averaged {75, 85} you would get 80. This might end up being input for another Reduce step with 50, and it would be a mistake to just naively average 80 and 50. The 80 has more weight.

Therefore, our Reduce step instead takes in {(80, 2), (50, 1)}, then sums the *weighted* temperatures. So it does $80 * 2 + 50 * 1$ and then divides by $(2 + 1)$ to get an average temperature of 70. It then emits (70, 3).

Another Reduce step might reduce {(25, 1), (100, 1)} to get (62.5, 2). If we reduce this with (70, 3) we get the final answer: (67, 5). In other words, the average temperature in this city for this year was 67 degrees.

We could do this in other ways, too. We could have just the city as the key, and the value be (Year, Temperature, Count). The Reduce step would do essentially the same thing, but would have to group by Year itself.

XI. Advanced Topics

In many cases, it's useful to think about what the Reduce step should do first, and then design the Map step around that. What data does Reduce need to have to do its job?

► Additional Studying

So, you've mastered this material and you want to learn even more? Okay. Here are some topics to get you started:

- **Bellman-Ford Algorithm:** Finds the shortest paths from a single node in a weighted directed graph with positive and negative edges.
- **Floyd-Warshall Algorithm:** Finds the shortest paths in a weighted graph with positive or negative weight edges (but no negative weight cycles).
- **Minimum Spanning Trees:** In a weighted, connected, undirected graph, a spanning tree is a tree that connects all the vertices. The minimum spanning tree is the spanning tree with minimum weight. There are various algorithms to do this.
- **B-Trees:** A self-balancing search tree (not a binary search tree) that is commonly used on disks or other storage devices. It is similar to a red-black tree, but uses fewer I/O operations.
- **A*:** Find the least-cost path between a source node and a goal node (or one of several goal nodes). It extends Dijkstra's algorithm and achieves better performance by using heuristics.
- **Interval Trees:** An extension of a balanced binary search tree, but storing intervals (low $\rightarrow$ high ranges) instead of simple values. A hotel could use this to store a list of all reservations and then efficiently detect who is staying at the hotel at a particular time.
- **Graph coloring:** A way of coloring the nodes in a graph such that no two adjacent vertices have the same color. There are various algorithms to do things like determine if a graph can be colored with only K colors.
- **P, NP, and NP-Complete:** P, NP, and NP-Complete refer to classes of problems. P problems are problems that can be quickly solved (where "quickly" means polynomial time). NP problems are those where, given a solution, the solution can be quickly verified. NP-Complete problems are a subset of NP problems that can all be reduced to each other (that is, if you found a solution to one problem, you could tweak the solution to solve other problems in the set in polynomial time).

It is an open (and very famous) question whether $P = NP$, but the answer is generally believed to be no.

- **Combinatorics and Probability:** There are various things you can learn about here, such as random variables, expected value, and n -choose- k .
- **Bipartite Graph:** A bipartite graph is a graph where you can divide its nodes into two sets such that every edge stretches across the two sets (that is, there is never an edge between two nodes in the same set). There is an algorithm to check if a graph is a bipartite graph. Note that a bipartite graph is equivalent to a graph that can be colored with two colors.
- **Regular Expressions:** You should know that regular expressions exist and what they can be used for (roughly). You can also learn about how an algorithm to match regular expressions would work. Some of the basic syntax behind regular expressions could be useful as well.

There is of course a great deal more to data structures and algorithms. If you're interested in exploring these topics more deeply, I recommend picking up the hefty *Introduction to Algorithms* ("CLRS" by Cormen, Leiserson, Rivest and Stein) or *The Algorithm Design Manual* (by Steven Skiena).

XII

Code Library

Certain patterns came up while implementing the code for this book. We've tried to generally include the full code for a solution with the solution, but in some cases it got quite redundant.

This appendix provides the code for a few of the most useful chunks of code.

The complete compilable solutions can be downloaded from CrackingTheCodingInterview.com.

XI

Code Library

Certain patterns came up while implementing the code for this book. We've tried to generally include the full code for a solution with the solution, but in some cases it got quite redundant.

This appendix provides the code for a few of the most useful chunks of code.

All code for the book can be downloaded from CrackingTheCodingInterview.com.

► HashMapList<T, E>

The HashMapList class is essentially shorthand for `HashMap<T, ArrayList<E>>`. It allows us to map from an item of type of `T` to an `ArrayList` of type `E`.

For example, we might want a data structure that maps from an integer to a list of strings. Ordinarily, we'd have to write something like this:

```
1  HashMap<Integer, ArrayList<String>> maplist =
2      new HashMap<Integer, ArrayList<String>>();
3  for (String s : strings) {
4      int key = computeValue(s);
5      if (!maplist.containsKey(key)) {
6          maplist.put(key, new ArrayList<String>());
7      }
8      maplist.get(key).add(s);
9  }
```

Now, we can just write this:

```
1  HashMapList<Integer, String> maplist = new HashMapList<Integer, String>();
2  for (String s : strings) {
3      int key = computeValue(s);
4      maplist.put(key, s);
5  }
```

It's not a big change, but it makes our code a bit simpler.

```
1  public class HashMapList<T, E> {
2      private HashMap<T, ArrayList<E>> map = new HashMap<T, ArrayList<E>>();
3
4      /* Insert item into list at key. */
5      public void put(T key, E item) {
6          if (!map.containsKey(key)) {
7              map.put(key, new ArrayList<E>());
8          }
9          map.get(key).add(item);
10 }
```

```

10     }
11
12     /* Insert list of items at key. */
13     public void put(T key, ArrayList<E> items) {
14         map.put(key, items);
15     }
16
17     /* Get list of items at key. */
18     public ArrayList<E> get(T key) {
19         return map.get(key);
20     }
21
22     /* Check if hashmaplist contains key. */
23     public boolean containsKey(T key) {
24         return map.containsKey(key);
25     }
26
27     /* Check if list at key contains value. */
28     public boolean containsKeyValue(T key, E value) {
29         ArrayList<E> list = get(key);
30         if (list == null) return false;
31         return list.contains(value);
32     }
33
34     /* Get the list of keys. */
35     public Set<T> keySet() {
36         return map.keySet();
37     }
38
39     @Override
40     public String toString() {
41         return map.toString();
42     }
43 }

```

► **TreeNode (Binary Search Tree)**

While it's perfectly fine—even good—to use the built-in binary tree class when possible, it's not always possible. In many questions, we needed access to the internals of the node or tree class (or needed to tweak these) and thus couldn't use the built-in libraries.

The `TreeNode` class supports a variety of functionality, much of which we wouldn't necessarily want for every question/solution. For example, the `TreeNode` class tracks the parent of the node, even though we often don't use it (or specifically ban using it).

For simplicity, we'd implemented this tree as storing integers for data.

```

1  public class TreeNode {
2      public int data;
3      public TreeNode left, right, parent;
4      private int size = 0;
5
6      public TreeNode(int d) {
7          data = d;
8          size = 1;
9      }

```

```
10
11 public void insertInOrder(int d) {
12     if (d <= data) {
13         if (left == null) {
14             setLeftChild(new TreeNode(d));
15         } else {
16             left.insertInOrder(d);
17         }
18     } else {
19         if (right == null) {
20             setRightChild(new TreeNode(d));
21         } else {
22             right.insertInOrder(d);
23         }
24     }
25     size++;
26 }
27
28 public int size() {
29     return size;
30 }
31
32 public TreeNode find(int d) {
33     if (d == data) {
34         return this;
35     } else if (d <= data) {
36         return left != null ? left.find(d) : null;
37     } else if (d > data) {
38         return right != null ? right.find(d) : null;
39     }
40     return null;
41 }
42
43 public void setLeftChild(TreeNode left) {
44     this.left = left;
45     if (left != null) {
46         left.parent = this;
47     }
48 }
49
50 public void setRightChild(TreeNode right) {
51     this.right = right;
52     if (right != null) {
53         right.parent = this;
54     }
55 }
56
57 }
```

This tree is implemented to be a binary search tree. However, you can use it for other purposes. You would just need to use the `setLeftChild`/`setRightChild` methods, or the `left` and `right` child variables. For this reason, we have kept these methods and variables public. We need this sort of access for many problems.

► LinkedListNode (Linked List)

Like the `TreeNode` class, we often needed access to the internals of a linked list in a way that the built-in linked list class wouldn't support. For this reason, we implemented our own class and used it for many problems.

```

1  public class LinkedListNode {
2      public LinkedListNode next, prev, last;
3      public int data;
4      public LinkedListNode(int d, LinkedListNode n, LinkedListNode p){
5          data = d;
6          setNext(n);
7          setPrevious(p);
8      }
9
10     public LinkedListNode(int d) {
11         data = d;
12     }
13
14     public LinkedListNode() { }
15
16     public void setNext(LinkedListNode n) {
17         next = n;
18         if (this == last) {
19             last = n;
20         }
21         if (n != null && n.prev != this) {
22             n.setPrevious(this);
23         }
24     }
25
26     public void setPrevious(LinkedListNode p) {
27         prev = p;
28         if (p != null && p.next != this) {
29             p.setNext(this);
30         }
31     }
32
33     public LinkedListNode clone() {
34         LinkedListNode next2 = null;
35         if (next != null) {
36             next2 = next.clone();
37         }
38         LinkedListNode head2 = new LinkedListNode(data, next2, null);
39         return head2;
40     }
41 }

```

Again, we've kept the methods and variables public because we often needed this access. This would allow the user to "destroy" the linked list, but we actually needed this sort of functionality for our purposes.

► Trie & TrieNode

The trie data structure is used in a few problems to make it easier to look up if a word is a prefix of any other words in a dictionary (or list of valid words). This is often used when we're recursively building words so that we can short circuit when the word is not valid.

```
1 public class Trie {
2     // The root of this trie.
3     private TrieNode root;
4
5     /* Takes a list of strings as an argument, and constructs a trie that stores
6      * these strings. */
7     public Trie(ArrayList<String> list) {
8         root = new TrieNode();
9         for (String word : list) {
10             root.addWord(word);
11         }
12     }
13
14
15     /* Takes a list of strings as an argument, and constructs a trie that stores
16      * these strings. */
17     public Trie(String[] list) {
18         root = new TrieNode();
19         for (String word : list) {
20             root.addWord(word);
21         }
22     }
23
24     /* Checks whether this trie contains a string with the prefix passed in as
25      * argument. */
26     public boolean contains(String prefix, boolean exact) {
27         TrieNode lastNode = root;
28         int i = 0;
29         for (i = 0; i < prefix.length(); i++) {
30             lastNode = lastNode.getChild(prefix.charAt(i));
31             if (lastNode == null) {
32                 return false;
33             }
34         }
35         return !exact || lastNode.terminates();
36     }
37
38     public boolean contains(String prefix) {
39         return contains(prefix, false);
40     }
41
42     public TrieNode getRoot() {
43         return root;
44     }
45 }
```

The Trie class uses the TrieNode class, which is implemented below.

```
1 public class TrieNode {
2     /* The children of this node in the trie.*/
3     private HashMap<Character, TrieNode> children;
4     private boolean terminates = false;
5
6     /* The character stored in this node as data.*/
7     private char character;
8
9     /* Constructs an empty trie node and initializes the list of its children to an
10      * empty hash map. Used only to construct the root node of the trie. */
```

```

11 public TrieNode() {
12     children = new HashMap<Character, TrieNode>();
13 }
14
15 /* Constructs a trie node and stores this character as the node's value.
16  * Initializes the list of child nodes of this node to an empty hash map. */
17 public TrieNode(char character) {
18     this();
19     this.character = character;
20 }
21
22 /* Returns the character data stored in this node. */
23 public char getChar() {
24     return character;
25 }
26
27 /* Add this word to the trie, and recursively create the child
28  * nodes. */
29 public void addWord(String word) {
30     if (word == null || word.isEmpty()) {
31         return;
32     }
33
34     char firstChar = word.charAt(0);
35
36     TrieNode child = getChild(firstChar);
37     if (child == null) {
38         child = new TrieNode(firstChar);
39         children.put(firstChar, child);
40     }
41
42     if (word.length() > 1) {
43         child.addWord(word.substring(1));
44     } else {
45         child.setTerminates(true);
46     }
47 }
48
49 /* Find a child node of this node that has the char argument as its data. Return
50  * null if no such child node is present in the trie. */
51 public TrieNode getChild(char c) {
52     return children.get(c);
53 }
54
55 /* Returns whether this node represents the end of a complete word. */
56 public boolean terminates() {
57     return terminates;
58 }
59
60 /* Set whether this node is the end of a complete word.*/
61 public void setTerminates(boolean t) {
62     terminates = t;
63 }
64 }

```

Hints

XIII

Interviewers usually don't just hand you a question and expect you to solve it. Rather, they will typically offer guidance when you're stuck, especially on the harder questions. It's impossible to totally simulate the interview experience in a book, but these hints are designed to get you closer.

Try to solve the questions independently when possible. But it's okay to look for some help when you are really struggling. Again, struggling is a normal part of the process.

I've organized the hints somewhat randomly here, such that all the hints for a problem aren't adjacent. This way you won't accidentally see the second hint when you're reading the first hint.

Hints for Data Structures

- #1.** 1.2 Describe what it means for two strings to be permutations of each other. Now, look at that definition you provided. Can you check the strings against that definition?
- #2.** 3.1 A stack is simply a data structure in which the most recently added elements are removed first. Can you simulate a single stack using an array? Remember that there are many possible solutions, and there are tradeoffs of each.
- #3.** 2.4 There are many solutions to this problem, most of which are equally optimal in runtime. Some have shorter, cleaner code than others. Can you brainstorm different solutions?
- #4.** 4.10 If T2 is a subtree of T1, how will its in-order traversal compare to T1's? What about its pre-order and post-order traversal?
- #5.** 2.6 A palindrome is something which is the same when written forwards and backwards. What if you reversed the linked list?
- #6.** 4.12 Try simplifying the problem. What if the path had to start at the root?
- #7.** 2.5 Of course, you could convert the linked lists to integers, compute the sum, and then convert it back to a new linked list. If you did this in an interview, your interviewer would likely accept the answer, and then see if you could do this without converting it to a number and back.
- #8.** 2.2 What if you knew the linked list size? What is the difference between finding the Kth-to-last element and finding the Xth element?
- #9.** 2.1 Have you tried a hash table? You should be able to do this in a single pass of the linked list.
- #10.** 4.8 If each node has a link to its parent, we could leverage the approach from question 2.7 on page 95. However, our interviewer might not let us make this assumption.
- #11.** 4.10 The in-order traversals won't tell us much. After all, every binary search tree with the same values (regardless of structure) will have the same in-order traversal. This is what in-order traversal means: contents are in-order. (And if it won't work in the specific case of a binary search tree, then it certainly won't work for a general binary tree.) The pre-order traversal, however, is much more indicative.
- #12.** 3.1 We could simulate three stacks in an array by just allocating the first third of the array to the first stack, the second third to the second stack, and the final third to the third stack. One might actually be much bigger than the others, though. Can we be more flexible with the divisions?

- #13. 2.6 Try using a stack.
- #14. 4.12 Don't forget that paths could overlap. For example, if you're looking for the sum 6, the paths 1->3->2 and 1->3->2->4->-6->2 are both valid.
- #15. 3.5 One way of sorting an array is to iterate through the array and insert each element into a new array in sorted order. Can you do this with a stack?
- #16. 4.8 The first common ancestor is the deepest node such that p and q are both descendants. Think about how you might identify this node.
- #17. 1.8 If you just cleared the rows and columns as you found 0s, you'd likely wind up clearing the whole matrix. Try finding the cells with zeros first before making any changes to the matrix.
- #18. 4.10 You may have concluded that if `T2.preorderTraversal()` is a substring of `T1.preorderTraversal()`, then T2 is a subtree of T1. This is almost true, except that the trees could have duplicate values. Suppose T1 and T2 have all duplicate values but different structures. The pre-order traversals will look the same even though T2 is not a subtree of T1. How can you handle situations like this?
- #19. 4.2 A minimal binary tree has about the same number of nodes on the left of each node as on the right. Let's focus on just the root for now. How would you ensure that about the same number of nodes are on the left of the root as on the right?
- #20. 2.7 You can do this in $O(A+B)$ time and $O(1)$ additional space. That is, you do not need a hash table (although you could do it with one).
- #21. 4.4 Think about the definition of a balanced tree. Can you check that condition for a single node? Can you check it for every node?
- #22. 3.6 We could consider keeping a single linked list for dogs and cats, and then iterating through it to find the first dog (or cat). What is the impact of doing this?
- #23. 1.5 Start with the easy thing. Can you check each of the conditions separately?
- #24. 2.4 Consider that the elements don't have to stay in the same relative order. We only need to ensure that elements less than the pivot must be before elements greater than the pivot. Does that help you come up with more solutions?
- #25. 2.2 If you don't know the linked list size, can you compute it? How does this impact the runtime?
- #26. 4.7 Build a directed graph representing the dependencies. Each node is a project and an edge exists from A to B if B depends on A (A must be built before B). You can also build it the other way if it's easier for you.
- #27. 3.2 Observe that the minimum element doesn't change very often. It only changes when a smaller element is added, or when the smallest element is popped.
- #28. 4.8 How would you figure out if p is a descendent of a node n?
- #29. 2.6 Assume you have the length of the linked list. Can you implement this recursively?
- #30. 2.5 Try recursion. Suppose you have two lists, A = 1->5->9 (representing 951) and B = 2->3->6->7 (representing 7632), and a function that operates on the remainder of the lists (5->9 and 3->6->7). Could you use this to create the sum method? What is the relationship between `sum(1->5->9, 2->3->6->7)` and `sum(5->9, 3->6->7)`?

- #31.** 4.10 Although the problem seems like it stems from duplicate values, it's really deeper than that. The issue is that the pre-order traversal is the same only because there are null nodes that we skipped over (because they're null). Consider inserting a placeholder value into the pre-order traversal string whenever you reach a null node. Register the null node as a "real" node so that you can distinguish between the different structures.
- #32.** 3.5 Imagine your secondary stack is sorted. Can you insert elements into it in sorted order? You might need some extra storage. What could you use for extra storage?
- #33.** 4.4 If you've developed a brute force solution, be careful about its runtime. If you are computing the height of the subtrees for each node, you could have a pretty inefficient algorithm.
- #34.** 1.9 If a string is a rotation of another, then it's a rotation at a particular point. For example, a rotation of `waterbottle` at character 3 means cutting `waterbottle` at character 3 and putting the right half (`erbottle`) before the left half (`wat`).
- #35.** 4.5 If you traversed the tree using an in-order traversal and the elements were truly in the right order, does this indicate that the tree is actually in order? What happens for duplicate elements? If duplicate elements are allowed, they must be on a specific side (usually the left).
- #36.** 4.8 Start with the root. Can you identify if root is the first common ancestor? If it is not, can you identify which side of root the first common ancestor is on?
- #37.** 4.10 Alternatively, we can handle this problem recursively. Given a specific node within T1, can we check to see if its subtree matches T2?
- #38.** 3.1 If you want to allow for flexible divisions, you can shift stacks around. Can you ensure that all available capacity is used?
- #39.** 4.9 What is the very first value that must be in each array?
- #40.** 2.1 Without extra space, you'll need $O(N^2)$ time. Try using two pointers, where the second one searches ahead of the first one.
- #41.** 2.2 Try implementing it recursively. If you could find the $(K-1)$ th to last element, can you find the Kth element?
- #42.** 4.11 Be very careful in this problem to ensure that each node is equally likely and that your solution doesn't slow down the speed of standard binary search tree algorithms (like `insert`, `find`, and `delete`). Also, remember that even if you assume that it's a balanced binary search tree, this doesn't mean that the tree is full/complete/perfect.
- #43.** 3.5 Keep the secondary stack in sorted order, with the biggest elements on the top. Use the primary stack for additional storage.
- #44.** 1.1 Try a hash table.
- #45.** 2.7 Examples will help you. Draw a picture of intersecting linked lists and two equivalent linked lists (by value) that do not intersect.
- #46.** 4.8 Try a recursive approach. Check if p and q are descendants of the left subtree and the right subtree. If they are descendants of different subtrees, then the current node is the first common ancestor. If they are descendants of the same subtree, then that subtree holds the first common ancestor. Now, how do you implement this efficiently?

- #47. 4.7 Look at this graph. Is there any node you can identify that will definitely be okay to build first?
- #48. 4.9 The root is the very first value that must be in every array. What can you say about the order of the values in the left subtree as compared to the values in the right subtree? Do the left subtree values need to be inserted before the right subtree?
- #49. 4.4 What if you could modify the binary tree node class to allow a node to store the height of its subtree?
- #50. 2.8 There are really two parts to this problem. First, detect if the linked list has a loop. Second, figure out where the loop starts.
- #51. 1.7 Try thinking about it layer by layer. Can you rotate a specific layer?
- #52. 4.12 If each path had to start at the root, we could traverse all possible paths starting from the root. We can track the sum as we go, incrementing `totalPaths` each time we find a path with our target sum. Now, how do we extend this to paths that can start anywhere? Remember: Just get a brute-force algorithm done. You can optimize later.
- #53. 1.3 It's often easiest to modify strings by going from the end of the string to the beginning.
- #54. 4.11 This is your own binary search tree class, so you can maintain any information about the tree structure or nodes that you'd like (provided it doesn't have other negative implications, like making `insert` much slower). In fact, there's probably a reason the interview question specified that it was your own class. You probably need to store some additional information in order to implement this efficiently.
- #55. 2.7 Focus first on just identifying if there's an intersection.
- #56. 3.6 Let's suppose we kept separate lists for dogs and cats. How would we find the oldest animal of any type? Be creative!
- #57. 4.5 To be a binary search tree, it's not sufficient that the `left.value <= current.value < right.value` for each node. Every node on the left must be less than the current node, which must be less than all the nodes on the right.
- #58. 3.1 Try thinking about the array as circular, such that the end of the array "wraps around" to the start of the array.
- #59. 3.2 What if we kept track of extra data at each stack node? What sort of data might make it easier to solve the problem?
- #60. 4.7 If you identify a node without any incoming edges, then it can definitely be built. Find this node (there could be multiple) and add it to the build order. Then, what does this mean for its outgoing edges?
- #61. 2.6 In the recursive approach (we have the length of the list), the middle is the base case: `isPermutation(middle)` is true. The node `x` to the immediate left of the middle: What can that node do to check if `x->middle->y` forms a palindrome? Now suppose that checks out. What about the previous node `a`? If `x->middle->y` is a palindrome, how can it check that `a->x->middle->y->b` is a palindrome?
- #62. 4.11 As a naive "brute force" algorithm, can you use a tree traversal algorithm to implement this algorithm? What is the runtime of this?

- #63.** 3.6 Think about how you'd do it in real life. You have a list of dogs in chronological order and a list of cats in chronological order. What data would you need to find the oldest animal? How would you maintain this data?
- #64.** 3.3 You will need to keep track of the size of each substack. When one stack is full, you may need to create a new stack.
- #65.** 2.7 Observe that two intersecting linked lists will always have the same last node. Once they intersect, all the nodes after that will be equal.
- #66.** 4.9 The relationship between the left subtree values and the right subtree values is, essentially, anything. The left subtree values could be inserted before the right subtree, or the reverse (right values before left), or any other ordering.
- #67.** 2.2 You might find it useful to return multiple values. Some languages don't directly support this, but there are workarounds in essentially any language. What are some of those workarounds?
- #68.** 4.12 To extend this to paths that start anywhere, we can just repeat this process for all nodes.
- #69.** 2.8 To identify if there's a cycle, try the "runner" approach described on page 93. Have one pointer move faster than the other.
- #70.** 4.8 In the more naive algorithm, we had one method that indicated if x is a descendent of n , and another method that would recurse to find the first common ancestor. This is repeatedly searching the same elements in a subtree. We should merge this into one `firstCommonAncestor` function. What return values would give us the information we need?
- #71.** 2.5 Make sure you have considered linked lists that are not the same length.
- #72.** 2.3 Picture the list $1 \rightarrow 5 \rightarrow 9 \rightarrow 12$. Removing 9 would make it look like $1 \rightarrow 5 \rightarrow 12$. You only have access to the 9 node. Can you make it look like the correct answer?
- #73.** 4.2 You could implement this by finding the "ideal" next element to add and repeatedly calling `insertValue`. This will be a bit inefficient, as you would have to repeatedly traverse the tree. Try recursion instead. Can you divide this problem into subproblems?
- #74.** 1.8 Can you use $O(N)$ additional space instead of $O(N^2)$? What information do you really need from the list of cells that are zero?
- #75.** 4.11 Alternatively, you could pick a random depth to traverse to and then randomly traverse, stopping when you get to that depth. Think this through, though. Does this work?
- #76.** 2.7 You can determine if two linked lists intersect by traversing to the end of each and comparing their tails.
- #77.** 4.12 If you've designed the algorithm as described thus far, you'll have an $O(N \log N)$ algorithm in a balanced tree. This is because there are N nodes, each of which is at depth $O(\log N)$ at worst. A node is touched once for each node above it. Therefore, the N nodes will be touched $O(\log N)$ time. There is an optimization that will give us an $O(N)$ algorithm.
- #78.** 3.2 Consider having each node know the minimum of its "substack" (all the elements beneath it, including itself).
- #79.** 4.6 Think about how an in-order traversal works and try to "reverse engineer" it.

- #80. 4.8 The `firstCommonAncestor` function could return the first common ancestor (if `p` and `q` are both contained in the tree), `p` if `p` is in the tree and not `q`, `q` if `q` is in the tree and not `p`, and `null` otherwise.
- #81. 3.3 Popping an element at a specific substack will mean that some stacks aren't at full capacity. Is this an issue? There's no right answer, but you should think about how to handle this.
- #82. 4.9 Break this down into subproblems. Use recursion. If you had all possible sequences for the left subtree and the right subtree, how could you create all possible sequences for the entire tree?
- #83. 2.8 You can use two pointers, one moving twice as fast as the other. If there is a cycle, the two pointers will collide. They will land at the same location at the same time. Where do they land? Why there?
- #84. 1.2 There is one solution that is $O(N \log N)$ time. Another solution uses some space, but is $O(N)$ time.
- #85. 4.7 Once you decide to build a node, its outgoing edge can be deleted. After you've done this, can you find other nodes that are free and clear to build?
- #86. 4.5 If every node on the left must be less than or equal to the current node, then this is really the same thing as saying that the biggest node on the left must be less than or equal to the current node.
- #87. 4.12 What work is duplicated in the current brute-force algorithm?
- #88. 1.9 We are essentially asking if there's a way of splitting the first string into two parts, `x` and `y`, such that the first string is `xy` and the second string is `yx`. For example, `x = wat` and `y = erbottle`. The first string is `xy = waterbottle`. The second string is `yx = erbottlewat`.
- #89. 4.11 Picking a random depth won't help us much. First, there's more nodes at lower depths than higher depths. Second, even if we re-balanced these probabilities, we could hit a "dead end" where we meant to pick a node at depth 5 but hit a leaf at depth 3. Re-balancing the probabilities is an interesting , though.
- #90. 2.8 If you haven't identified the pattern of where the two pointers start, try this: Use the linked list `1->2->3->4->5->6->7->8->9->?`, where the `?` links to another node. Try making the `?` the first node (that is, the 9 points to the 1 such that the entire linked list is a loop). Then make the `?` the node 2. Then the node 3. Then the node 4. What is the pattern? Can you explain why this happens?
- #91. 4.6 Here's one step of the logic: The successor of a specific node is the leftmost node of the right subtree. What if there is no right subtree, though?
- #92. 1.6 Do the easy thing first. Compress the string, then compare the lengths.
- #93. 2.7 Now, you need to find where the linked lists intersect. Suppose the linked lists were the same length. How could you do this?

- #94.** 4.12 Consider each path that starts from the root (there are N such paths) as an array. What our brute-force algorithm is really doing is taking each array and finding all contiguous subsequences that have a particular sum. We're doing this by computing all subarrays and their sums. It might be useful to just focus on this little subproblem. Given an array, how would you find all contiguous subsequences with a particular sum? Again, think about the duplicated work in the brute-force algorithm.
- #95.** 2.5 Does your algorithm work on linked lists like $9 \rightarrow 7 \rightarrow 8$ and $6 \rightarrow 8 \rightarrow 5$? Double check that.
- #96.** 4.8 Careful! Does your algorithm handle the case where only one node exists? What will happen? You might need to tweak the return values a bit.
- #97.** 1.5 What is the relationship between the "insert character" option and the "remove character" option? Do these need to be two separate checks?
- #98.** 3.4 The major difference between a queue and a stack is the order of elements. A queue removes the oldest item and a stack removes the newest item. How could you remove the oldest item from a stack if you only had access to the newest item?
- #99.** 4.11 A naive approach that many people come up with is to pick a random number between 1 and 3. If it's 1, return the current node. If it's 2, branch left. If it's 3, branch right. This solution doesn't work. Why not? Is there a way you can adjust it to make it work?
- #100.** 1.7 Rotating a specific layer would just mean swapping the values in four arrays. If you were asked to swap the values in two arrays, could you do this? Can you then extend it to four arrays?
- #101.** 2.6 Go back to the previous hint. Remember: There are ways to return multiple values. You can do this with a new class.
- #102.** 1.8 You probably need some data storage to maintain a list of the rows and columns that need to be zeroed. Can you reduce the additional space usage to $O(1)$ by using the matrix itself for data storage?
- #103.** 4.12 We are looking for subarrays with sum `targetSum`. Observe that we can track in constant time the value of `runningSumi`, where this is the sum from element 0 through element i . For a subarray of element i through element j to have sum `targetSum`, `runningSumi-1 + targetSum` must equal `runningSumj` (try drawing a picture of an array or a number line). Given that we can track the `runningSum` as we go, how can we quickly look up the number of indices i where the previous equation is true?
- #104.** 1.9 Think about the earlier hint. Then think about what happens when you concatenate `erbottlewat` to itself. You get `erbottlewaterbottlewat`.
- #105.** 4.4 You don't need to modify the binary tree class to store the height of the subtree. Can your recursive function compute the height of each subtree while also checking if a node is balanced? Try having the function return multiple values.
- #106.** 1.4 You do not have to—and should not—generate all permutations. This would be very inefficient.
- #107.** 4.3 Try modifying a graph search algorithm to track the depth from the root.
- #108.** 4.12 Try using a hash table that maps from a `runningSum` value to the number of elements with this `runningSum`.

- #109.** 2.5 For the follow-up question: The issue is that when the linked lists aren't the same length, the head of one linked list might represent the 1000's place while the other represents the 10's place. What if you made them the same length? Is there a way to modify the linked list to do that, without changing the value it represents?
- #110.** 1.6 Be careful that you aren't repeatedly concatenating strings together. This can be very inefficient.
- #111.** 2.7 If the two linked lists were the same length, you could traverse forward in each until you found an element in common. Now, how do you adjust this for lists of different lengths?
- #112.** 4.11 The reason that the earlier solution (picking a random number between 1 and 3) doesn't work is that the probabilities for the nodes won't be equal. For example, the root will be returned with probability $\frac{1}{3}$, even if there are 50+ nodes in the tree. Clearly, not all the nodes have probability $\frac{1}{3}$, so these nodes won't have equal probability. We can resolve this one issue by picking a random number between 1 and `size_of_tree` instead. This only resolves the issue for the root, though. What about the rest of the nodes?
- #113.** 4.5 Rather than validating the current node's value against `leftTree.max` and `rightTree.min`, can we flip around the logic? Validate the left tree's nodes to ensure that they are smaller than `current.value`.
- #114.** 3.4 We can remove the oldest item from a stack by repeatedly removing the newest item (inserting those into the temporary stack) until we get down to one element. Then, after we've retrieved the newest item, putting all the elements back. The issue with this is that doing several pops in a row will require $O(N)$ work each time. Can we optimize for scenarios where we might do several pops in a row?
- #115.** 4.12 Once you've solidified the algorithm to find all contiguous subarrays in an array with a given sum, try to apply this to a tree. Remember that as you're traversing and modifying the hash table, you may need to "reverse the damage" to the hash table as you traverse back up.
- #116.** 4.2 Imagine we had a `createMinimalTree` method that returns a minimal tree for a given array (but for some strange reason doesn't operate on the root of the tree). Could you use this to operate on the root of the tree? Could you write the base case for the function? Great! Then that's basically the entire function.
- #117.** 1.1 Could a bit vector be useful?
- #118.** 1.3 You might find you need to know the number of spaces. Can you just count them?
- #119.** 4.11 The issue with the earlier solution is that there could be more nodes on one side of a node than the other. So, we need to weight the probability of going left and right based on the number of nodes on each side. How does this work, exactly? How can we know the number of nodes?
- #120.** 2.7 Try using the difference between the lengths of the two linked lists.
- #121.** 1.4 What characteristics would a string that is a permutation of a palindrome have?
- #122.** 1.2 Could a hash table be useful?
- #123.** 4.3 A hash table or array that maps from level number to nodes at that level might also be useful.

- #124.** 4.4 Actually, you can just have a single `checkHeight` function that does both the height computation and the balance check. An integer return value can be used to indicate both.
- #125.** 4.7 As a totally different approach: Consider doing a depth-first search starting from an arbitrary node. What is the relationship between this depth-first search and a valid build order?
- #126.** 2.2 Can you do it iteratively? Imagine if you had two pointers pointing to adjacent nodes and they were moving at the same speed through the linked list. When one hits the end of the linked list, where will the other be?
- #127.** 4.1 Two well-known algorithms can do this. What are the tradeoffs between them?
- #128.** 4.5 Think about the `checkBST` function as a recursive function that ensures each node is within an allowable (`min`, `max`) range. At first, this range is infinite. When we traverse to the left, the `min` is negative infinity and the `max` is `root.value`. Can you implement this recursive function and properly adjust these ranges as you traverse the tree?
- #129.** 2.7 If you move a pointer in the longer linked list forward by the difference in lengths, you can then apply a similar approach to the scenario when the linked lists are equal.
- #130.** 1.5 Can you do all three checks in a single pass?
- #131.** 1.2 Two strings that are permutations should have the same characters, but in different orders. Can you make the orders the same?
- #132.** 1.1 Can you solve it in $O(N \log N)$ time? What might a solution like that look like?
- #133.** 4.7 Pick an arbitrary node and do a depth-first search on it. Once we get to the end of a path, we know that this node can be the last one built, since no nodes depend on it. What does this mean about the nodes right before it?
- #134.** 1.4 Have you tried a hash table? You should be able to get this down to $O(N)$ time.
- #135.** 4.3 You should be able to come up with an algorithm involving both depth-first search and breadth-first search.
- #136.** 1.4 Can you reduce the space usage by using a bit vector?



Hints for Concepts and Algorithms

- #137.** 5.1 Break this into parts. Focus first on clearing the appropriate bits.
- #138.** 8.9 Try the Base Case and Build approach.
- #139.** 6.9 Given a specific door x , on which rounds will it be toggled (open or closed)?
- #140.** 11.5 What does the interviewer mean by a pen? There are a lot of different types of pens. Make a list of potential questions you would want to ask.
- #141.** 7.11 This is not as complicated as it sounds. Start by making a list of the key objects in the system, then think about how they interact.
- #142.** 9.6 First, start with making some assumptions. What do and don't you have to build?
- #143.** 5.2 To wrap your head around the problem, try thinking about how you'd do it for integers.
- #144.** 8.6 Try the Base Case and Build approach.
- #145.** 5.7 Swapping each pair means moving the even bits to the left and the odd bits to the right. Can you break this problem into parts?
- #146.** 6.10 Solution 1: Start with a simple approach. Can you just divide up the bottles into groups? Remember that you can't re-use a test strip once it is positive, but you can reuse it as long as it's negative.
- #147.** 5.4 Get Next: Start with a brute force solution for each.
- #148.** 8.14 Can we just try all possibilities? What would this look like?
- #149.** 6.5 Play around with the jugs of water, pouring water back and forth, and see if you can measure anything other than 3 quarts or 5 quarts. That's a start.
- #150.** 8.7 Approach 1: Suppose you had all permutations of abc . How can you use that to get all permutations of $abcd$?
- #151.** 5.5 Reverse engineer this, starting from the outermost layer to the innermost layer.
- #152.** 8.1 Approach this from the top down. What is the very last hop the child made?
- #153.** 7.1 Note that a "card deck" is very broad. You might want to think about a reasonable scope to the problem.
- #154.** 6.7 Observe that each family will have exactly one girl.
- #155.** 8.13 Will sorting the boxes help in any way?

- #156.** 6.8 This is really an algorithm problem, and you should approach it as such. Come up with a brute force, compute the worst-case number of drops, then try to optimize that.
- #157.** 6.4 In what cases will they not collide?
- #158.** 9.6 We've assumed that the rest of the eCommerce system is already handled, and we just need to deal with the analytics part of sales rank. We can get notified somehow when a purchase occurs.
- #159.** 5.3 Start with a brute force solution. Can you try all possibilities?
- #160.** 6.7 Think about writing each family as a sequence of Bs and Gs.
- #161.** 8.8 You could handle this by just checking to see if there are duplicates before printing them (or adding them to a list). You can do this with a hash table. In what case might this be okay? In what case might it not be a very good solution?
- #162.** 9.7 Will this application be write-heavy or read-heavy?
- #163.** 6.10 Solution 1: There is a relatively simple approach that works in 28 days, in the worst case. There are better approaches though.
- #164.** 11.5 Consider the scenario of a pen for children. What does this mean? What are the different use cases?
- #165.** 9.8 Scope the problem well. What will and won't you tackle as part of this system?
- #166.** 8.5 Think about multiplying 8 by 9 as counting the number of cells in a matrix with width 8 and height 9.
- #167.** 5.2 In a number like .893 (in base 10), what does each digit signify? What then does each digit in .10010 signify in base 2?
- #168.** 8.14 We can think about each possibility as each place where we can put parentheses. This means around each operator, such that the expression is split at the operator. What is the base case?
- #169.** 5.1 To clear the bits, create a "bit mask" that looks like a series of 1s, then 0s, then 1s.
- #170.** 8.3 Start with a brute force algorithm.
- #171.** 6.7 You can attempt this mathematically, although the math is pretty difficult. You might find it easier to estimate it up to families of, say, 6 children. This won't give you a good mathematical proof, but it might point you in the right direction of what the answer might be.
- #172.** 6.9 In which cases would a door be left open at the end of the process?
- #173.** 5.2 A number such as .893 (in base 10) indicates $8 * 10^{-1} + 9 * 10^{-2} + 3 * 10^{-3}$. Translate this system into base 2.
- #174.** 8.9 Suppose we had all valid ways of writing two pairs of parentheses. How could we use this to get all valid ways of writing three pairs?
- #175.** 5.4 Get Next: Picture a binary number—something with a bunch of 1s and 0s spread out throughout the number. Suppose you flip a 1 to a 0 and a 0 to a 1. In what case will the number get bigger? In what case will it get smaller?

- #176.** 9.6 Think about what sort of expectations on freshness and accuracy of data is expected. Does the data always need to be 100% up to date? Is the accuracy of some products more important than others?
- #177.** 10.2 How do you check if two words are anagrams of each other? Think about what the definition of “anagram” is. Explain it in your own words.
- #178.** 8.1 If we knew the number of paths to each of the steps before step 100, could we compute the number of steps to 100?
- #179.** 7.8 Should white pieces and black pieces be the same class? What are the pros and cons of this?
- #180.** 9.7 Observe that there is a lot of data coming in, but people probably aren’t reading the data very frequently.
- #181.** 6.2 Calculate the probability of winning the first game and winning the second game, then compare them.
- #182.** 10.2 Two words are anagrams if they contain the same characters but in different orders. How can you put characters in order?
- #183.** 6.10 Solution 2: Why do we have such a time lag between tests and results? There’s a reason the question isn’t phrased as just “minimize the number of rounds of testing.” The time lag is there for a reason.
- #184.** 9.8 How evenly do you think traffic is distributed? Do all documents get roughly the same age of traffic? Or is it likely there are some very popular documents?
- #185.** 8.7 Approach 1: The permutations of abc represent all ways of ordering abc. Now, we want to create all orderings of abcd. Take a specific ordering of abcd, such as bdca. This bdca string represents an ordering of abc, too: Remove the d and you get bca. Given the string bca, can you create all the “related” orderings that include d, too?
- #186.** 6.1 You can only use the scale once. This means that all, or almost all, of the bottles must be used. They also must be handled in different ways or else you couldn’t distinguish between them.
- #187.** 8.9 We could try generating the solution for three pairs by taking the list of two pairs of parentheses and adding a third pair. We’d have to add the third paren before, around, and after. That is: ()<SOLUTION>, (<SOLUTION>), <SOLUTION>(). Will this work?
- #188.** 6.7 Logic might be easier than math. Imagine we wrote every birth into a giant string of Bs and Gs. Note that the groupings of families are irrelevant for this problem. What is the probability of the next character added to the string being a B versus a G?
- #189.** 9.6 Purchases will occur very frequently. You probably want to limit database writes.
- #190.** 8.8 If you haven’t solved 8.7 yet, do that one first.
- #191.** 6.10 Solution 2: Consider running multiple tests at once.
- #192.** 7.6 A common trick when solving a jigsaw puzzle is to separate edge and non-edge pieces. How will you represent this in an object-oriented manner?
- #193.** 10.9 Start with a naive solution. (But hopefully not too naive. You should be able to use the fact that the matrix is sorted.)

- #194.** 8.13 We can sort the boxes by any dimension in descending order. This will give us a partial order for the boxes, in that boxes later in the array must appear before boxes earlier in the array.
- #195.** 6.4 The only way they won't collide is if all three are walking in the same direction. What's the probability of all three walking clockwise?
- #196.** 10.11 Imagine the array were sorted in ascending order. Is there any way you could "fix it" to be sorted into alternating peaks and valleys?
- #197.** 8.14 The base case is when we have a single value, 1 or 0.
- #198.** 7.3 Scope the problem first and make a list of your assumptions. It's often okay to make reasonable assumptions, but you need to make them explicit.
- #199.** 9.7 The system will be write-heavy: Lots of data being imported, but it's rarely being read.
- #200.** 8.7 Approach 1: Given a string such as `bca`, you can create all permutations of `abcd` that have `{a, b, c}` in the order `bca` by inserting `d` into each possible location: `dbca`, `bdca`, `bcda`, `bcad`. Given all permutations of `abc`, can you then create all permutations of `abcd`?
- #201.** 6.7 Observe that biology hasn't changed; only the conditions under which a family stops having kids has changed. Each pregnancy has a 50% odds of being a boy and a 50% odds of being a girl.
- #202.** 5.5 What does it mean if `A & B == 0`?
- #203.** 8.5 If you wanted to count the cells in an `8x9` matrix, you could count the cells in a `4x9` matrix and then double it.
- #204.** 8.3 Your brute force algorithm probably ran in $O(N)$ time. If you're trying to beat that runtime, what runtime do you think you will get to? What sorts of algorithms have that runtime?
- #205.** 6.10 Solution 2: Think about trying to figure out the bottle, digit by digit. How can you detect the first digit in the poisoned bottle? What about the second digit? The third digit?
- #206.** 9.8 How will you handle generating URLs?
- #207.** 10.6 Think about merge sort versus quick sort. Would one of them work well for this purpose?
- #208.** 9.6 You also want to limit joins because they can be very expensive.
- #209.** 8.9 The problem with the solution suggested by the earlier hint is that it might have duplicate values. We could eliminate this by using a hash table.
- #210.** 11.6 Be careful about your assumptions. Who are the users? Where are they using this? It might seem obvious, but the real answer might be different.
- #211.** 10.9 We can do a binary search in each row. How long will this take? How can we do better?
- #212.** 9.7 Think about things like how you're going to get the bank data (will it be pulled or pushed?), what features the system will support, etc.
- #213.** 7.7 As always, scope the problem. Are "friendships" mutual? Do status messages exist? Do you support group chat?
- #214.** 8.13 Try to break it down into subproblems.

- #215.** 5.1 It's easy to create a bit mask of 0s at the beginning or end. But how do you create a bit mask with a bunch of zeroes in the middle? Do it the easy way: Create a bit mask for the left side and then another one for the right side. Then you can merge those.
- #216.** 7.11 What is the relationship between files and directories?
- #217.** 8.1 We can compute the number of steps to 100 by the number of steps to 99, 98, and 97. This corresponds to the child hopping 1, 2, or 3 steps at the end. Do we add those or multiply them? That is: Is it $f(100) = f(99) + f(98) + f(97)$ or $f(100) = f(99) * f(98) * f(97)$?
- #218.** 6.6 This is a logic problem, not a clever word problem. Use logic/math/algorithms to solve it.
- #219.** 10.11 Try walking through a sorted array. Can you just swap elements until you have fixed the array?
- #220.** 11.5 Have you considered both intended uses (writing, etc.) and unintended use? What about safety? You would not want a pen for children to be dangerous.
- #221.** 6.10 Solution 2: Be very careful about edge cases. What if the third digit in the bottle number matches the first or second digit?
- #222.** 8.8 Try getting the count of each character. For example, ABCAAC has 3 As, 2 Cs, and 1 B.
- #223.** 9.6 Don't forget that a product can be listed under multiple categories.
- #224.** 8.6 You can easily move the smallest disk from one tower to another. It's also pretty easy to move the smallest two disks from one tower to another. Can you move the smallest three disks?
- #225.** 11.6 In a real interview, you would also want to discuss what sorts of test tools we have available.
- #226.** 5.3 Flipping a 0 to a 1 can merge two sequences of 1s—but only if the two sequences are separated by only one 0.
- #227.** 8.5 Think about how you might handle this for odd numbers.
- #228.** 7.8 What class should maintain the score?
- #229.** 10.9 If you're considering a particular column, is there a way to quickly eliminate it (in some cases at least)?
- #230.** 6.10 Solution 2: You can run an additional day of testing to check digit 3 in a different way. But again, be very careful about edge cases here.
- #231.** 10.11 Note that if you ensure the peaks are in place, the valleys will be, too. Therefore, your iteration to fix the array can skip over every other element.
- #232.** 9.8 If you generate URLs randomly, do you need to worry about collisions (two documents with the same URL)? If so, how can you handle this?
- #233.** 6.8 As a first approach, you might try something like binary search. Drop it from the 50th floor, then the 75th, then the 88th, and so on. The problem is that if the first egg drops at the 50th floor, then you'll need to start dropping the second egg starting from the 1st floor and going up. This could take, at worst, 50 drops (the 50th floor drop, the 1st floor drop, the 2nd floor drop, and up through the 49th floor drop). Can you beat this?

- #234.** 8.5 If there's duplicated work across different recursive calls, can you cache it?
- #235.** 10.7 Would a bit vector help?
- #236.** 9.6 Where would it be appropriate to cache data or queue up tasks?
- #237.** 8.1 We multiply the values when it's "we do this then this." We add them when it's "we do this or this."
- #238.** 7.6 Think about how you might record the position of a piece when you find it. Should it be stored by row and location?
- #239.** 6.2 To calculate the probability of winning the second game, start with calculating the probability of making the first hoop, the second hoop, and not the third hoop.
- #240.** 8.3 Can you solve the problem in $O(\log N)$?
- #241.** 6.10 Solution 3: Think about each test strip as being a binary indicator for poisoned vs. non-poisoned.
- #242.** 5.4 Get Next: If you flip a 1 to a 0 and a 0 to a 1, it will get bigger if the 0->1 bit is more significant than the 1->0 bit. How can you use this to create the next biggest number (with the same number of 1s)?
- #243.** 8.9 Alternatively, we could think about doing this by moving through the string and adding left and right parens at each step. Will this eliminate duplicates? How do we know if we can add a left or right paren?
- #244.** 9.6 Depending on what assumptions you made, you might even be able to do without a database at all. What would this mean? Would it be a good idea?
- #245.** 7.7 This is a good problem to think about the major system components or technologies that would be useful.
- #246.** 8.5 If you're doing $9*7$ (both odd numbers), then you could do $4*7$ and $5*7$.
- #247.** 9.7 Try to reduce unnecessary database queries. If you don't need to permanently store the data in the database, you might not need it in the database at all.
- #248.** 5.7 Can you create a number that represents just the even bits? Then can you shift the even bits over by one?
- #249.** 6.10 Solution 3: If each test strip is a binary indicator, can we map , integer keys to a set of 10 binary indicators such that each key has a unique configuration (mapping)?
- #250.** 8.6 Think about moving the smallest disk from tower $X=0$ to tower $Y=2$ using tower $Z=1$ as a temporary holding spot as having a solution for $f(1, X=0, Y=2, Z=1)$. Moving the smallest two disks is $f(2, X=0, Y=2, Z=1)$. Given that you have a solution for $f(1, X=0, Y=2, Z=1)$ and $f(2, X=0, Y=2, Z=1)$, can you solve $f(3, X=0, Y=2, Z=1)$?
- #251.** 10.9 Since each column is sorted, you know that the value can't be in this column if it's smaller than the min value in this column. What else does this tell you?
- #252.** 6.1 What happens if you put one pill from each bottle on the scale? What if you put two pills from each bottle on the scale?
- #253.** 10.11 Do you necessarily need the arrays to be sorted? Can you do it with an unsorted array?

- #254.** 10.7 To do it with less memory, can you try multiple passes?
- #255.** 8.8 To get all permutations with 3 As, 2 Cs, and 1 B, you need to first pick a starting character: A, B, or C. If it's an A, then you need all permutations with 2 As, 2 Cs, and 1 B.
- #256.** 10.5 Try modifying binary search to handle this.
- #257.** 11.1 There are two mistakes in this code.
- #258.** 7.4 Does the parking lot have multiple levels? What "features" does it support? Is it paid? What types of vehicles?
- #259.** 9.5 You may need to make some assumptions (in part because you don't have an interviewer here). That's okay. Make those assumptions explicit.
- #260.** 8.13 Think about the first decision you have to make. The first decision is which box will be at the bottom.
- #261.** 5.5 If $A \& B == 0$, then it means that A and B never have a 1 at the same spot. Apply this to the equation in the problem.
- #262.** 8.1 What is the runtime of this method? Think carefully. Can you optimize it?
- #263.** 10.2 Can you leverage a standard sorting algorithm?
- #264.** 6.9 Note: If an integer x is divisible by a , and $b = x / a$, then x is also divisible by b . Does this mean that all numbers have an even number of factors?
- #265.** 8.9 Adding a left or right paren at each step will eliminate duplicates. Each substring will be unique at each step. Therefore, the total string will be unique.
- #266.** 10.9 If the value x is smaller than the start of the column, then it also can't be in any columns to the right.
- #267.** 8.7 Approach 1: You can create all permutations of $abcd$ by computing all permutations of abc and then inserting d into each possible location within those.
- #268.** 11.6 What are the different features and uses we would want to test?
- #269.** 5.2 How would you get the first digit in $.893$? If you multiplied by 10, you'd shift the values over to get 8.93 . What happens if you multiply by 2?
- #270.** 9.2 To find the connection between two nodes, would it be better to do a breadth-first search or depth-first search? Why?
- #271.** 7.7 How will you know if a user signs offline?
- #272.** 8.6 Observe that it doesn't really matter which tower is the source, destination, or buffer. You can do $f(3, X=0, Y=2, Z=1)$ by first doing $f(2, X=0, Y=1, Z=2)$ (moving two disks from tower 0 to tower 1, using tower 2 as a buffer), then moving disk 3 from tower 0 to tower 2, then doing $f(2, X=1, Y=2, Z=0)$ (moving two disks from tower 1 to tower 2, using tower 0 as a buffer). How does this process repeat?
- #273.** 8.4 How can you build all subsets of $\{a, b, c\}$ from the subsets of $\{a, b\}$?
- #274.** 9.5 Think about how you could design this for a single machine. Would you want a hash table? How would that work?
- #275.** 7.1 How, if at all, will you handle aces?

- #276.** 9.7 As much work as possible should be done asynchronously.
- #277.** 10.11 Suppose you had a sequence of three elements $\{0, 1, 2\}$, in any order. Write out all possible sequences for those elements and how you can fix them to make 1 the peak.
- #278.** 8.7 Approach 2: If you had all permutations of two-character substrings, could you generate all permutations of three-character substrings?
- #279.** 10.9 Think about the previous hint in the context of rows.
- #280.** 8.5 Alternatively, if you're doing $9 * 7$, you could do $4 * 7$, double that, and then add 7.
- #281.** 10.7 Try using one pass to get it down to a range of values, and then a second pass to find a specific value.
- #282.** 6.6 Suppose there were exactly one blue-eyed person. What would that person see? When would they leave?
- #283.** 7.6 Which will be the easiest pieces to match first? Can you start with those? Which will be the next easiest, once you've nailed those down?
- #284.** 6.2 If two events are mutually exclusive (they can never occur simultaneously), you can add their probabilities together. Can you find a set of mutually exclusive events that represent making two out of three hoops?
- #285.** 9.2 A breadth-first search is probably better. A depth-first search can wind up going on a long path, even though the shortest path is actually very short. Is there a modification to a breadth-first search that might be even faster?
- #286.** 8.3 Binary search has a runtime of $O(\log N)$. Can you apply a form of binary search to the problem?
- #287.** 7.12 In order to handle collisions, the hash table should be an array of linked lists.
- #288.** 10.9 What would happen if we tried to keep track of this using an array? What are the pros and cons of this?
- #289.** 10.8 Can you use a bit vector?
- #290.** 8.4 Anything that is a subset of $\{a, b\}$ is also a subset of $\{a, b, c\}$. Which sets are subsets of $\{a, b, c\}$ but not $\{a, b\}$?
- #291.** 10.9 Can we use the previous hints to move up, down, left, and right around the rows and columns?
- #292.** 10.11 Revisit the set of sequences for $\{0, 1, 2\}$ that you just wrote out. Imagine there are elements before the leftmost element. Are you sure that the way you swap the elements won't invalidate the previous part of the array?
- #293.** 9.5 Can you combine a hash table and a linked list to get the best of both worlds?
- #294.** 6.8 It's actually better for the first drop to be a bit lower. For example, you could drop at the 10th floor, then the 20th floor, then the 30th floor, and so on. The worst case here will be 19 drops (10, 20, ..., 100, 91, 92, ..., 99). Can you beat that? Try not randomly guessing at different solutions. Rather, think deeper. How is the worst case defined? How does the number of drops of each egg factor into that?